Adevar Labs

Prepared for:

Reflect Money

October 08, 2026

RLP Oracle Stacking Audit Report

Version: after fix review

Table of Contents

1. Executive Summary 4

About RLP Oracle Stacking 4

Audit Summary 4

Overall Security Posture 4

After Fix Review Summary 5

Before Fix Review Summary 5

Launch Recommendations 5

Audit Scope 5

2. Assumptions and Considerations 6

Audit Assumptions 6

3. Severity Definitions 7

Impact 7

Likelihood 7

Severity Classification Matrix 7

4. Findings 9

M01: A matured cooldown never expires, so an LP can withdraw ahead of a slash 9

L01: Two Assets on one feed can store a book the tighter Asset cannot read 11

L02: Opposite inverse flags on one feed compose to $1.00 13

L03: A deposit into a pool slashed to zero mints at the launch rate and hands most of the deposit to existing LPs 14

About Us 16

About Adevar Labs 16

Audit Methodology 16

Confidentiality Notice 18

© 2026 Adevar Labs. All rights reserved.

Legal Disclaimer 18

© 2026 Adevar Labs. All rights reserved.

1. Executive Summary

About RLP Oracle Stacking

RLP prices an Asset from a chain of oracle legs. This review covers how those legs are composed, posted, and read. It also covers withdrawal cooldowns and how deposits are priced after a slash.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status.

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	1	1	0	0
Low	3	2	1	0

Enhancement opportunities: 0.

Overall Security Posture

We found no Critical or High issues. The stacking path is in place: a straight leg multiplies, an inverse leg divides, one v3 feed maps to one stored price, and each read checks that price against the reading Asset's own confidence bound.

One issue is rated Medium. A matured withdrawal cooldown never expires, so a liquidity provider (LP) can hold an exit open indefinitely and withdraw at the pre-slash value once a loss is visible on chain, leaving the slash to the LPs who stay.

Three issues are rated Low. Two depend on how the admin configures the oracle chain. The first arises when two Assets share one feed with different confidence bounds: a report accepted through the looser Asset can leave the tighter Asset without a readable price until a newer report arrives. The second arises when one feed is set as both a straight and an inverse leg: the legs cancel to $1.00, and a band around $1 accepts the chain. The Launch Recommendations below cover the part of the first that remains after the fixes. The third affects a pool that a slash has emptied: the next deposit mints LP tokens at the launch rate while earlier LP tokens are still outstanding, so most of that deposit goes to existing LPs.

© 2026 Adevar Labs. All rights reserved.

After Fix Review Summary

The Reflect team fixed three of the four issues and partially fixed the fourth.

A withdrawal now waits until any loss on the protected proxy vault has been slashed, so an LP can no longer exit ahead of the slash, and a matured cooldown that goes unclaimed returns its LP tokens to the owner. A deposit into a pool that a slash has emptied now fails. An oracle chain whose legs cancel each other is now rejected when it is saved.

Each Asset now keeps its own Chainlink Data Streams price. Pyth prices are still stored once per feed, so two Assets that share a Pyth feed with different confidence bounds can still leave the tighter one without a readable price until a newer update arrives.

Before Fix Review Summary

We reviewed oracle stacking on RLP. An Asset can name several legs. More than one Asset can name the same feed. **post_oracle_price** writes one account, and the Asset reads it back.

We checked Pyth, Chainlink Data Streams v3, v7 rates, Doppler, and Data Feeds. We checked straight and inverse legs, shared feeds, confidence bounds, staleness, and the price band. We also checked withdrawal cooldowns and how deposits are priced after a slash.

One issue is rated Medium and three are rated Low. Details are in the Findings section.

Launch Recommendations

- Set a tight **min** and **max** around the real composed price. A 10x band does not catch a flipped **inverse** near $1.
- Share a Pyth feed between two Assets only when both use the same confidence bound.
- Slash promptly after a loss appears on the protected proxy vault. Withdrawals from the pool now wait for that slash.

Audit Scope

- **Repository:** https://github.com/palindrome-eng/reflect-program-library
- **Before Fix Review Commit Hash:** **ccc4fb9a89dcda3249f09a7caf1f99cecb5b9adc**
- **After Fix Review Commit Hash:** **faf13ce7bdd960e28dac2a6e7c566c545cb50122**
- **Files/Modules in Scope:**
  - **rlp/programs/rlp/src/**

© 2026 Adevar Labs. All rights reserved.

2. Assumptions and Considerations

Audit Assumptions

1. A 10x price band cannot detect a wrong inverse on a rate near 1.0

add_asset and **update_oracle** accept a live composed price only when **min <= price <= max** and **max** is below 10x **min**. That check catches a wrong hop whose price is far from the intended one. It does not catch a flipped **inverse** bit on a feed near 1.0. Both **r** and **1/r** sit inside any band that wide. Only the admin sets the bit. This review treats a well-set chain as the intended case.

2. The admin and the oracle signers are trusted

The admin sets the chain, the **inverse** bit, the decimals, the confidence bound, and the staleness window. Pyth and the Chainlink DON sign the reports this program stores. This review assumes that the admin is honest and careful, and that those signers publish the price they claim to publish. A bad chain, a flipped **inverse**, or a wrong scale set by that admin is a centralization risk, not a defect in the program. A signer that attests to a false price is the same.

© 2026 Adevar Labs. All rights reserved.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help convey both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

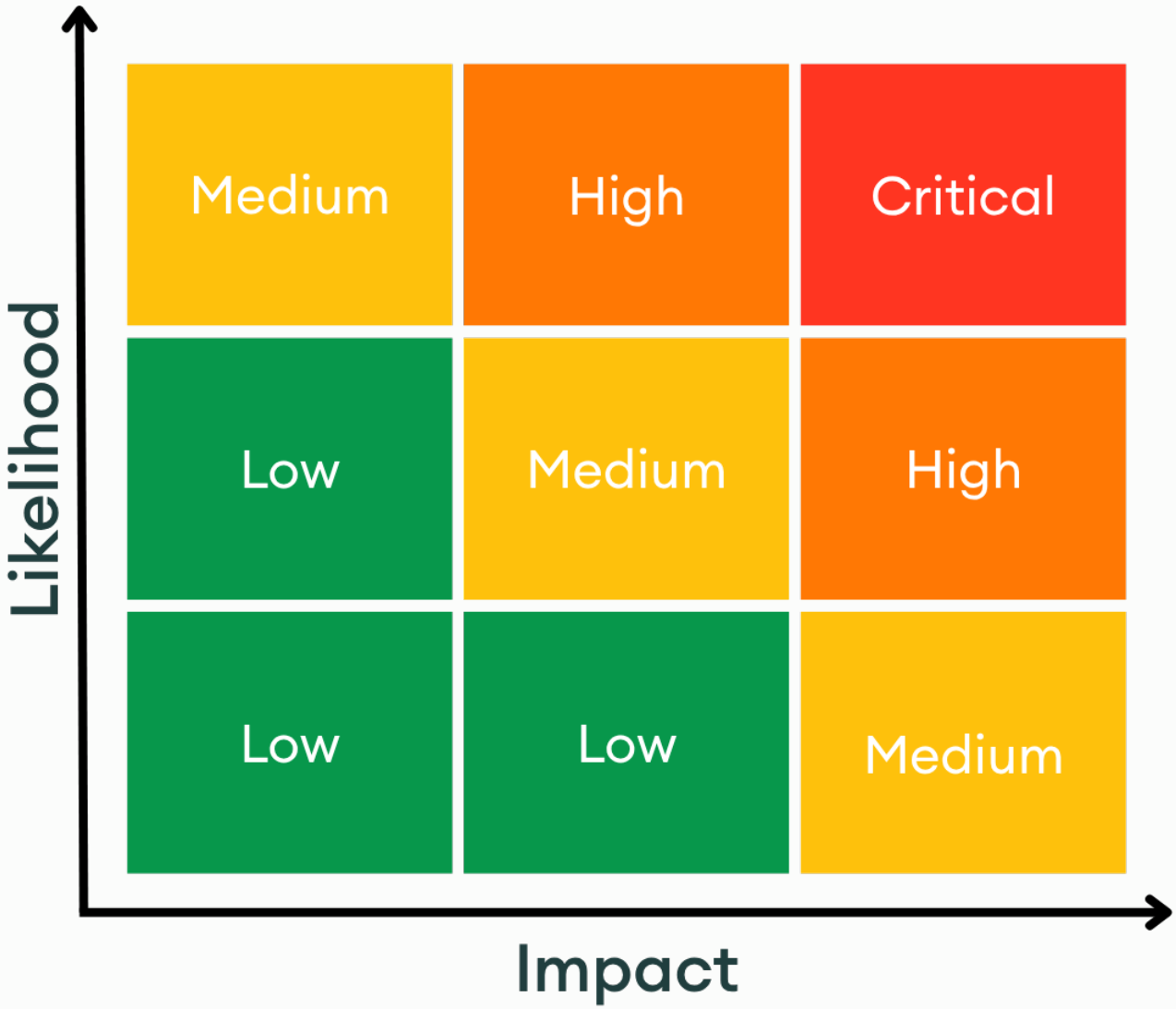
By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

© 2026 Adevar Labs. All rights reserved.

- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

© 2026 Adevar Labs. All rights reserved.

4. Findings

M01: A matured cooldown never expires, so an LP can withdraw ahead of a slash



LOCATION

reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/instructions/user/withdraw.rs:L63-L66

RELEVANT SNIPPET

>_ withdraw.rs — RUST

```
61:  let clock = Clock::get()?;
62:
63:  require!(
64:      clock.unix_timestamp as u64 >= cooldown.unlock_ts,
65:      RlpError::CooldownInForce
66:  );
67:
68:  require!(
```

OTHER LOCATIONS

- reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/states/cooldown.rs:L20-L29

DESCRIPTION

The cooldown exists "to prevent frontrunning of senior-tranche loss coverage" (README): a liquidity provider (LP) must wait out **cooldown_duration** between requesting a withdrawal and withdrawing. The **locked_amount** fix for H01 in the RLP Protocol Audit Report (June 19, 2026) fixed the withdrawal amount at request time, so an empty cooldown can no longer be topped up later. A matured cooldown, however, still never expires.

1. **request_withdrawal** stores **locked_amount**, calls **Cooldown::lock**, and moves the LP tokens into the cooldown ATA. The LP tokens stay in **lp_token_mint.supply**.
2. **Cooldown::lock** sets only **unlock_ts = now + cooldown_duration**. **Cooldown** has no expiry field.
3. **withdraw** checks only that the current time has reached **cooldown.unlock_ts**, then pays **reserve.amount * locked_amount / lp_token_supply** at the current reserve.

Nothing ends a matured cooldown. An LP requests once, waits out **cooldown_duration**, and then holds an exit usable at any later moment. Their locked LP tokens keep their pro-rata share meanwhile, so keeping a matured cooldown open costs nothing.

© 2026 Adevar Labs. All rights reserved.

When the protected proxy vault's booked claims exceed the value it holds (a NAV loss), the loss is visible on chain before it is covered. Covering it takes a separate **slash** transaction, which only an account with the CRANK role can send. The LP withdraws first at the pre-slash value, and the slash falls on the LPs who stay.

For example, in a $1,000,000 pool where Alice holds 40%, a $300,000 NAV loss lets Alice exit with $400,000 instead of $280,000, and the remaining LPs absorb her $120,000 share.

RECOMMENDATION

Give each cooldown a claim window: **withdraw** refuses after **unlock_ts + CLAIM_WINDOW**, and a new instruction returns the LP tokens from an expired cooldown to their owner.

RUST

```rust
let expires_at = cooldown.unlock_ts.checked_add(CLAIM_WINDOW).ok_or(RlpError::MathOverflow)?;
require!(clock.unix_timestamp as u64 <= expires_at, RlpError::CooldownExpired);
```

With a 24 h cooldown and a 1 h window, at most 1/25 of an LP's stake can be withdrawable at any moment. Add a test asserting that **withdraw** one second after **unlock_ts + CLAIM_WINDOW** fails.

FIX REVIEW NOTES

Fixed in **faf13ce**, which replaces the first fix in **8910b83**. **withdraw** now refuses to pay out while the protected proxy vault shows a loss that a slash could still cover from the pool, so an LP can no longer exit ahead of the slash. A matured cooldown that is not claimed within one further cooldown length returns its LP tokens to the owner.

© 2026 Adevar Labs. All rights reserved.

L01: Two Assets on one feed can store a book the tighter Asset cannot read



LOCATION

reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/states/asset.rs:L249-L252

RELEVANT SNIPPET

>_ asset.rs — RUST

```rust
// A provider that publishes no interval cannot be bounded. Reject a stored bound on
// such a leg at write time, not here.
if let Some(bps) = price.confidence_bps {
    let bound = confidence_bound_bps(leg.max_confidence_bps);
    require!(bps <= bound, RlpError::OracleConfidenceTooWide);
}

Ok(price)
```

DESCRIPTION

One Chainlink v3 feed has one stored price. The account is a PDA derived from the seed **stream** and the feed id. Every Asset that names that feed reads that same account.

post_oracle_price takes one Asset. The program reads the confidence limit from the leg whose oracle address is that account. It does not read the other Assets that share the feed.

Use two Assets on one feed. Both declare the same decimals. Tight allows a 2% spread. Loose allows a 10% spread. A signed report has a 5% spread. A post through Tight fails. A post through Loose succeeds. The saved spread is 5%.

Tight then reads the same account. 5% is above its 2% limit, so the read in **asset.rs** fails with **OracleConfidenceTooWide**. An older report that was inside 2% cannot replace the saved one, because **post_oracle_price** only accepts a report newer than the stored one. Tight has no price until a later report is both newer and inside 2%.

If the admin sets no limit, the program uses 2%. The highest an admin can set is 10%. The report must still pass the limit of the Asset in the transaction.

Sharing one feed is allowed. If both Assets use the same limit, a report one accepts is a report the other accepts. A v7 rate report stores no spread, so this path does not apply to a redemption-rate leg.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Do not let one Asset overwrite the shared price with a report another Asset on that feed cannot read. Key the price account by Asset as well as feed id, or require the saved report to pass every Asset that names the feed.

FIX REVIEW NOTES

Partially fixed in **d1fce8e**. Each Asset now has its own Chainlink Data Streams price account, derived from the asset mint and the feed id, so a report posted through one Asset no longer replaces the price another Asset reads. The Pyth price account is still derived from the feed id alone, so two Assets on one Pyth feed with different confidence bounds still share one stored price. Adding the asset mint to the Pyth seed would close this remaining case.

© 2026 Adevar Labs. All rights reserved.

L02: Opposite inverse flags on one feed compose to $1.00



LOCATION

reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/helpers/oracle/compose.rs:L42-L46

RELEVANT SNIPPET

>_ compose.rs — RUST

```
40:  require!(scaled != 0, RlpError::PriceError);
41:
42:  acc = if *inverse {
43:      mul_div_floor(acc, COMPOSITION_SCALE, scaled)?
44:  } else {
45:      mul_div_floor(acc, scaled, COMPOSITION_SCALE)?
46:  };
47:
48:  if let Some(leg_bps) = leg.confidence_bps {
```

DESCRIPTION

update_oracle stores the legs the admin sends. It does not compare their oracle addresses or their feed ids.

compose_oracle_legs multiplies by a straight leg and divides by an inverse leg.

Use one feed whose price is $76,822. The admin sets leg 0 to that feed with **inverse = false**. The admin sets leg 1 to the same feed with **inverse = true**. The second leg divides the first. The composed price is **10^18** with exponent **-18**, which is $1.00. A band around $1 accepts the chain.

Each leg adds its confidence. Two copies of a tight feed stay inside the chain bound.

RECOMMENDATION

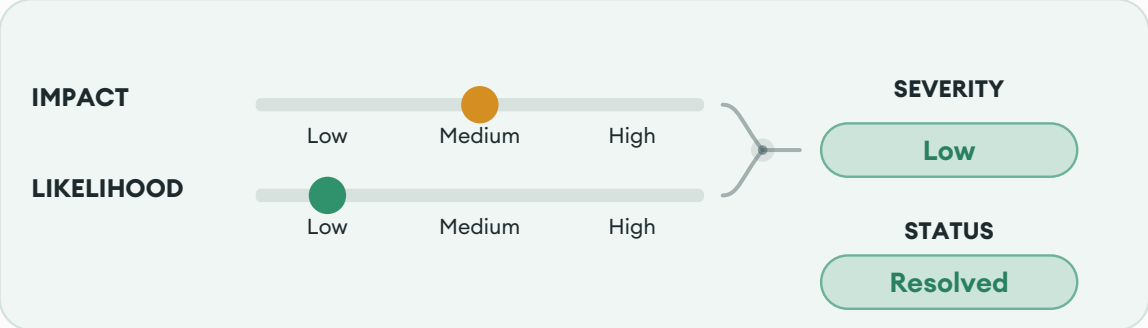
Reject two legs that share an oracle address and use different inverse flags. Also reject two legs that share a feed id and use different inverse flags.

FIX REVIEW NOTES

Fixed in **50f398e**. **add_asset** and **update_oracle** now reject a chain in which two legs share an oracle address or a feed id and carry opposite **inverse** flags.

© 2026 Adevar Labs. All rights reserved.

L03: A deposit into a pool slashed to zero mints at the launch rate and hands most of the deposit to existing LPs



LOCATION

reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/states/liquidity_pool.rs:L234-L251

RELEVANT SNIPPET

>_ liquidity_pool.rs — RUST

```
232:     deposit_value: PreciseNumber,
233: ) -> Result<u64> {
234:     let pool_value_is_zero = total_pool_value
235:         .to_imprecise()
236:         .map_or(true, |v| v == 0);
237:
238:     let lp_tokens_to_mint = if lp_token.supply == 0 || pool_value_is_zero {
239:         let lp_decimals = lp_token.decimals as u32;
240:         let scale_down_precise = PreciseNumber::new(10u128.pow(PRECISION - lp_decimals
    ))
241:             .ok_or(crate::errors::RlpError::MathOverflow)?;
242:
243:         deposit_value
244:             .checked_div(&scale_down_precise)
245:             .ok_or(crate::errors::RlpError::MathOverflow)?
246:             .floor()
247:             .ok_or(crate::errors::RlpError::MathOverflow)?
248:             .to_imprecise()
249:             .ok_or(crate::errors::RlpError::MathOverflow)?
250:             .try_into()
251:             .map_err(|_| crate::errors::RlpError::MathOverflow)?
252:     } else {
253:         let lp_supply_precise = PreciseNumber::new(lp_token.supply as u128)
```

OTHER LOCATIONS

- reflect-program-library/ccc4fb9a/rlp/programs/rlp/src/instructions/slash/slash.rs:L120-L128

DESCRIPTION

calculate_lp_tokens_on_deposit mints at the launch rate, **deposit_value / 10^(PRECISION - lp_decimals)**, whenever **pool_value_is_zero**, even while LP tokens are outstanding. The **slash** instruction bounds its **amount** only by the net asset value (NAV) loss, the amount by which the protected proxy vault's booked claims exceed the value it holds, and by the reserve balance. A NAV loss at least as large as the reserve therefore lets a slash leave the pool at zero value with every LP token still circulating. The next depositor then owns **minted / (supply + minted)** of the pool: $1,000

© 2026 Adevar Labs. All rights reserved.

deposited into mainnet pool 4 (156,990 LP tokens outstanding) leaves the depositor with $6.33, and existing LPs can withdraw the rest.

RECOMMENDATION

Refuse a deposit when the pool value is zero and the LP token supply exceeds the dead shares that **initialize_lp** mints, so a wiped pool stays closed instead of repricing at the launch rate.

```rust
let dead_shares = 10u64.pow(lp_token.decimals as u32 - 3);
require!(!(pool_value_is_zero && lp_token.supply > dead_shares), RlpError::PoolWipedOut);
```

FIX REVIEW NOTES

Fixed in **c2af4fa**. A deposit now fails with **PoolWipedOut** when the pool value is zero and the LP token supply is above the dead shares minted at initialization, so a pool emptied by a slash stays closed.

© 2026 Adevar Labs. All rights reserved.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Finally, we examine external integrations with other Solana projects and verify that inputs and return values are handled properly.

© 2026 Adevar Labs. All rights reserved.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited, we may also include a detailed proof-of-concept exploit with code snippets and instructions on how the attack could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

© 2026 Adevar Labs. All rights reserved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 Adevar Labs. All rights reserved.